Code-Layout, Readability and Reusability

Date	20 October 2026
Team ID	Individual project (no team)
Project Name	Personalized Travel Itinerary Planner
Maximum Marks	5 Marks

Self-assessment of the codebase structure, readability and reusability. Repository: https://github.com/niniwritescode/travel-itinerary-planner

Code Layout Checklist

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing used throughout the code	Yes	4 spaces in Python, 2 spaces in JSX and CSS.
2	Proper File Structure	Files and folders are logically organised	Yes	backend/ and frontend/ are separate; frontend code is in src/.
3	Meaningful Variable Names	Variables reflect their purpose	Yes	e.g. TripRequest, itinerary, saved, busy.
4	Function / Method Names	Functions are descriptively named	Yes	ask_groq, build_prompt, save_trip, list_trips, generate, toText.
5	Code Comments	Inline and block comments explain logic	Partial	Key backend functions are commented with their purpose; the frontend has fewer comments.
6	Modular Design	Code is split into reusable functions / modules	Partial	Backend logic is in helper functions inside one file (main.py); the frontend has a separate Itinerary component.
7	No Redundant Code	Duplicate or unused code is removed	Yes	ask_groq() is reused by two endpoints; no dead code.
8	Error Handling	Exceptions and errors are handled gracefully	Partial	Backend validates input and returns HTTP errors; the frontend shows messages on failure. No retries or response schema validation yet.

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	ask_groq()	Python / Groq SDK	POST /itinerary and POST /recommend	High
2	build_prompt()	Python	Itinerary generation (can be extended for regeneration)	Medium
3	TripRequest model	Python / Pydantic	Both AI endpoints	Medium
4	post() helper	JavaScript	Itinerary, suggestions and save-trip calls	High
5	Itinerary component	React (JSX)	Generated trip and saved trip views	High

Overall Code Quality Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clear frontend/backend split and small files.
Readability	4	Descriptive names and short functions.

Aspect	Rating (1-5)	Comments
Reusability	3	Good helper reuse; backend could be split into routers and services.
Documentation / Comments	3	README and API docs page are available; more inline comments in the frontend would help.
Overall Score	3.5	Solid for a first full-stack version; planned refactor into modules.